KI-System-Prompt & Vollständige Assembler-Referenz: Fließkomma- & Festkomma-Arithmetik auf dem 16-Bit Relaiscomputer (CLACK-F16 & Q8.8)

Autoren / Referenz-Architektur: Clack-BASIC Cross-Compiler (`pkg/codegen/float_f16.go` & `pkg/codegen/fixed_q88.go`)

Auftraggeber: @peter:latentforge.dev

Zielgruppe: KI-Systeme (LLMs / Coding-Agenten) und Entwickler, die Software direkt in **16-Bit Relais-Assembler** (`clack asm` / Maschinencode) schreiben und dabei die Fließkomma-Berechnungen (`CLACK-F16` und `Q8.8`) exakt wie der Compiler umsetzen müssen.

---

1. System-Prompt-Block (Direkt kopierbar für KIs)

> **ROLLE & AUFGABE FÜR DIE KI:**
>
> Du schreibst direkten 16-Bit Assembler- und Maschinencode für unseren elektromechanischen 16-Bit Relaiscomputer (1024 Worte RAM, Adressen `0` bis `1023` , 19 Opcodes, Ein-Register-Akkumulator `Accu`).
>
> Befolge bei **allen** Fließkomma- (`CLACK-F16` , Zuse-Stil `S1-E7-M8`) und Festkomma-Berechnungen (`Q8.8` / `FIXED8`) **exakt** die in diesem Dokument spezifizierten Datenformate, Calling-Conventions, Selbstmodifikations-Muster (Dynamic Operand Patching) und Assembler-Subroutinen des Clack-BASIC Compilers.
>
> Erfinde **keine** imaginären Opcodes (es gibt kein `SUB` , kein `XOR` , kein `MUL` , kein `DIV` und keinen Barrel-Shifter in Hardware; `SHR` , `SHL` und `NOT` belegen im Assembler-Abbild **2 Worte**: Opcode + `0x0000` , nur `RET` und `NOP` belegen **1 Wort**!).

---

2. Hardware-Architektur, Speicherkarte & Befehlssatz (ISA)

2.1 Zentrale Architektur-Regeln (KRITISCH FÜR ASSEMBLER-KIs!)

1. **Ein-Register-Maschine (`Accu`):** Alle ALU-Operationen arbeiten ausschließlich auf dem 16-Bit Akkumulator (`Accu`).
2. **Kein Register-zu-Register-ADD (`ADD` hat nur `#Immediate`):** Der Befehl `ADD #Imm` (`0x3440`) addiert das **unmittelbar im nächsten Speicherwort (`PC+1`) stehende 16-Bit-Literal** zum `Accu` . Um eine Variable `[varB]` zu `[varA]` zu addieren, **muss** der Wert von `[varB]` per `STAM` in das Operandenwort des `ADD` -Befehls gepatcht werden (*Dynamic Operand Patching* / Selbstmodifizierender Code).
3. **Zweierkomplement-Subtraktion (`A - B`):** Da kein `SUB` -Opcode existiert, wird `A - B` immer als `A + (~B + 1)` berechnet (`LDAM B` → `NOT` → `ADD 1` → Patch in `ADD` -Operand → `LDAM A` → `ADD 0`).
4. **1-Bit-Schieberegister:** `SHL` (`0x35C0`) und `SHR` (`0x3580`) schieben den `Accu` um **exakt 1 Bit** (`SHL` schiebt Bit 15 in das Carry-Flag; `SHR` ist ein logischer Rechtsshift mit `0` -Nachschub in Bit 15). Ein Shift um 8 Bit (`>> 8` oder `<< 8`) wird durch **8 aufeinanderfolgende `SHR` - bzw. `SHL` -Befehle** (oder eine Zählschleife bei dynamischer Shiftweite) realisiert.
5. **Wortlänge der Befehle im Speicher (WICHTIG für Adress-Patching!):**

- **Fast alle Befehle belegen 2 Speicherworte** (`Wort 0` = Opcode, `Wort 1` = 16-Bit Operand).
- **Auch `NOT` , `SHR` und `SHL` werden vom Compiler und Assembler als 2-Wort-Befehle emittiert** (`Opcode` an `PC` , `0x0000` an `PC+1`)!
- **Nur `RET` (`0x6000`) und `NOP` (`0xF000`) belegen exakt 1 Speicherwort!**

1. **1-Level Hardware-Return-Register (`SR`):** `CALL target` speichert `PC+2` in einem **einzigen** Hardware-Register `SR` (kein Stack im RAM!). Laufzeit-Subroutinen dürfen daher **niemals** ein weiteres `CALL` ausführen (stattdessen Tail-Call per `BA`, wie bei `__f16_sub` → `__f16_add`).

2.2 Vollständige Opcode-Tabelle (19 Hardware-Befehle)

Mnemonic	Opcode (Hex)	Worte	Operation / Semantik	Flags / Hinweise
`NOP`	`0xF000`	**1**	Keine Operation	1 Wort!
`LDA #imm`	`0x1400`	2	`Accu = imm`	Lädt 16-Bit Konstante
`LDAM addr`	`0x1800`	2	`Accu = RAM[addr]`	Direkter Speicherzugriff
`LDAMM ptr`	`0x1C00`	2	`Accu = RAM[RAM[ptr]]`	Indirekter Lesezugriff (Zeiger)
`STAM addr`	`0x2800`	2	`RAM[addr] = Accu`	Speichert `Accu` (`Accu` bleibt erhalten!)
`STAMM ptr`	`0x2C00`	2	`RAM[RAM[ptr]] = Accu`	Indirekter Schreibzugriff (Zeiger)
`ADD #imm`	`0x3440`	2	`{Carry, Accu} = Accu + imm`	Setzt Carry (`BC`), Negativ (`BN`), Zero (`BZ`)
`AND #imm`	`0x3480`	2	`Accu = Accu & imm`	Bitweises UND
`OR #imm`	`0x34C0`	2	`Accu = Accu | imm`	Bitweises ODER
`NOT`	`0x3540`	2	`Accu = ~Accu`	2 Worte (`0x3540` , `0x0000`)
`SHR`	`0x3580`	2	`Accu = Accu >> 1` (logisch, Bit 15 = `0`)	2 Worte (`0x3580` , `0x0000`)
`SHL`	`0x35C0`	2	`{Carry, Accu} = Accu << 1` (Bit 0 = `0`)	2 Worte (`0x35C0` , `0x0000`); Carry = altes Bit 15
`BA target`	`0x4000`	2	`PC = target`	Unbedingter Sprung (`HALT` = `BA $`)
`BC target`	`0x4008`	2	`if (Carry == 1) PC = target`	Bei `A + (~B + 1)` gilt: `Carry == 1` $\Longleftrightarrow A \geq B$ (unsigned)
`BN target`	`0x4010`	2	`if (Accu[15] == 1) PC = target`	Sprung wenn negativ (`Bit 15 == 1`)
`BZ target`	`0x4018`	2	`if (Accu == 0x0000) PC = target`	Sprung wenn Null
`BE target`	`0x4020`	2	`if (Accu[0] == 0) PC = target`	Sprung wenn gerade (`LSB == 0`)
`CALL target`	`0x5000`	2	`SR = PC + 2; PC = target`	1-Level Hardware-Return-Register `SR`
`RET`	`0x6000`	**1**	`PC = SR`	**1 Wort!** Rücksprung aus Subroutine

2.3 Speicherkarte (Memory Layout des Compilers)

Adressbereich (`SCREEN 32`)	**Adressbereich (`SCREEN 64`)**	**Verwendung**
`0 .. 1`	`0 .. 1`	Initialer Boot-Sprung: `BA 4` (`0x4000` , `0x0004`)
`2`	`2`	Hardware Output Port (8-Bit LED-Anzeige)
`3`	`3`	Hardware Input Port (8-Bit Schalter)
`4 .. 919`	`4 .. 733`	Programmcode + FPU-Laufzeitroutinen + `DATA` -Tabellen
`920 .. 951` (`0x0398..0x03B7`)	`734 .. 755`	Globale Variablen & FPU-Arbeitsregister (`__r0` , `__r1` , ...)
`952 .. 959` (`0x03B8..0x03BF`)	`756 .. 767`	Temporäre Auswertungszellen (`temp0` .. `temp7` für Klammerausdrücke)
`960 .. 1023` (`0x03C0..0x03FF`)	`768 .. 1023` (`0x0300..0x03FF`)	Video-RAM (LED-Matrix Framebuffer)

---

3. Grundlegende Assembler-Idiome des Compilers (PFLICHTMUSTER)

Da die ALU als zweiten Operanden bei `ADD` , `AND` , `OR` nur ein Immediate-Wort im Befehl selbst akzeptiert, erzeugt der Compiler für Variablen-Operationen selbstmodifizierenden Code.

Idiom 1: Dynamisches `ADD` zweier Speicherzellen (`Accu = RAM[A] + RAM[B]`)

Zwischen `STAM patch_addr` (2 Worte) und `LDAM A` (2 Worte) bis zum Operandenwort von `ADD` (+1 Wort) liegen exakt **5 Adressworte (`g.pc + 5`)**:

```
    LDAM varB            ; [PC+0, PC+1] Lade B in Accu
    STAM __patch_add_op  ; [PC+2, PC+3] Schreibe B direkt in das Operandenwort bei PC+7!
    LDAM varA            ; [PC+4, PC+5] Lade A in Accu
    ADD 0x0000           ; [PC+6] Opcode 0x3440 | [PC+7 = __patch_add_op] Operand (wurde mit B überschr
```

Im Assembler (`clack asm`): Wenn du mit Labels arbeitest, kannst du direkt auf `label + 1` patchen oder das Label vor das `DATA` -Wort bzw. die Instruktion setzen. In reinem Assembler ohne Ausdrucks-Arithmetik im Operanden berechnet man entweder die feste Adresse oder legt ein Label auf die Instruktion und nutzt eine bekannte Adresse. Noch einfacher in handgeschriebenem `clack asm` :

```
    ; Im clack-basic Compiler wird exakt auf die Operanden-Adresse (Befehlsadresse + 1) gepatcht.
    ; Wenn der Befehl ADD an Adresse P steht, liegt sein 16-Bit-Immediate-Operand an Adresse P+1!
```

Idiom 2: Dynamische Subtraktion (`Accu = RAM[A] - RAM[B]`)

```
    LDAM varB            ; [PC+0, PC+1] Lade Subtrahend B
    NOT                  ; [PC+2, PC+3] ~B (Einerkomplement)
    ADD 1                ; [PC+4, PC+5] -B = ~B + 1 (Zweierkomplement)
    STAM __patch_sub_op  ; [PC+6, PC+7] Patche -B in das Operandenwort bei PC+11
    LDAM varA            ; [PC+8, PC+9] Lade Minuend A
    ADD 0x0000           ; [PC+10, PC+11 = __patch_sub_op] Berechnet A + (-B) = A - B
```

Idiom 3: Konstante Subtraktion (`Accu = Accu - K`)

Für eine bekannte Konstante K wird im Compiler einfach `ADD (65536 - K)` emittiert:

- `Accu - 1` : `ADD 0xFFFF`
- `Accu - 9` : `ADD 0xFFF7`
- `Accu - 63` : `ADD 0xFFC1`
- `Accu - 71` : `ADD 0xFFB9`

- `Accu - 78`: `ADD 0xFFB2`
- `Accu - 8` im Exponenten (Bits 14..8): `ADD 0xF800`

Idiom 4: Vorzeichen-Toggle / Bit-15-XOR ohne `XOR`-Befehl!

Da **Bit 15 das höchste Bit** des 16-Bit-Wortes ist, erzeugt eine Addition von `0x8000` **keinen Übertrag in niedrigere Bits (Bits 14..0 bleiben völlig unverändert)**, während Bit 15 exakt invertiert wird (`0 + 1 = 1`, `1 + 1 = 0` mit ignoriertem Carry-Out)!

- **Vorzeichenbit invertieren (`x = -x` in `CLACK-F16`):**

```asm
ADD 0x8000 ; Invertiert Bit 15 (XOR 0x8000), Bits 14..0 bleiben unangetastet!
```

- **Zwei Vorzeichenbits `signA` (`0x0000`/`0x8000`) und `signB` (`0x0000`/`0x8000`) per XOR verknüpfen:**

Einfach `signA + signB` mit `ADD` addieren!

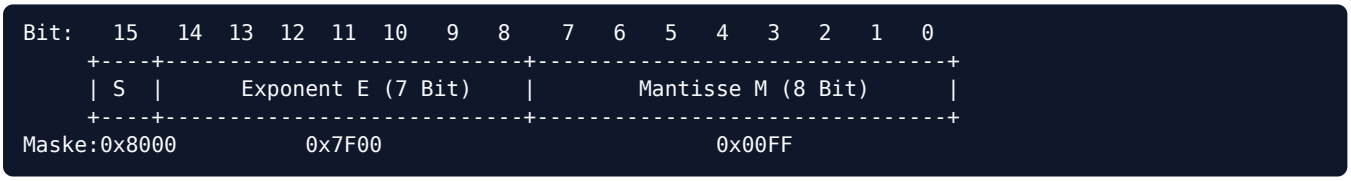
- `0x0000 + 0x0000 = 0x0000` (gleiches Vorzeichen → Ergebnis `0`, `BZ` springt!)
- `0x8000 + 0x8000 = 0x0000` (gleiches Vorzeichen → Ergebnis `0`, `BZ` springt!)
- `0x8000 + 0x0000 = 0x8000` (unterschiedliches Vorzeichen → Ergebnis `0x8000`)

---

4. Format 1: Echtes Gleitkomma `CLACK-F16` (`S1-E7-M8` Zuse-Stil)

4.1 Bit-Layout & Mathematische Definition

Das im Compiler (`pkg/codegen/float_f16.go`) implementierte 16-Bit Gleitkommaformat `CLACK-F16` orientiert sich an Konrad Zuses Gleitkomma-Architektur (ohne NaN/Denormal-Ballast) und ist **perfekt auf Byte-Grenzen ausgerichtet**:



- **Bit 15 (`S`, Maske `0x8000`):** Vorzeichenbit (`0` = positiv, `1` = negativ).
- **Bits 14..8 (`E`, Maske `0x7F00`):** 7-Bit Exponent mit **Bias `63` (`0x3F`)**, Wertebereich normalisiert `E = 1 .. 127`.
- **Bits 7..0 (`M`, Maske `0x00FF`):** 8-Bit gebrochene Mantisse $f \in \{0..255\}$. Die **implizite führende `1`** steht auf Bit 8 (`0x0100`). Entpackt ist die 9-Bit Mantisse also immer `MA = (Wort & 0x00FF) | 0x0100` $\in [256 .. 511]$.
- **Formel für den Zahlenwert ($E > 0$):**

$$\text{Wert} = (-1)^S \cdot \left(1 + \frac{M}{256}\right) \cdot 2^{E - 63} = (-1)^S \cdot \left(\frac{256 + M}{256}\right) \cdot 2^{E - 63}$$

- **Kanonische Null (`0.0`):** `0x0000` (Jedes Wort mit `(Wort & 0x7FFF) == 0`, also `E == 0` und `M == 0`, wird beim Lesen als `0.0` interpretiert; Rückgaben erzeugen stets kanonisch `0x0000`).
- **Sättigung bei Overflow / Division durch 0:**
- Positiver Maximalwert (`F16MaxPos`): `0x7FFF` ($S=0, E=127, M=255 \approx +3.682 \times 10^{19}$)
- Negativer Maximalwert (`F16MaxNeg`): `0xFFFF` ($S=1, E=127, M=255 \approx -3.682 \times 10^{19}$)
- **Kleinster positiver Wert (`E=1, M=0`):** `0x0100` ($= 2^{-62} \approx 2.168 \times 10^{-19}$). Bei $E \le 0$ erfolgt **Underflow nach `0x0000`**.

4.2 Wichtige `CLACK-F16` Referenz-Konstanten (Hex-Tabelle)

Dezimalwert	Rechnung *(S, E, M)*	Hex-Wort (`uint16`)	Dezimal (`uint16`)
`0.0`	Kanonische Null	`0x0000`	`0`
`0.1`	*E=59 (-4), M=154*	`0x3B9A`	`15258`
`0.25`	*E=61 (-2), M=0*	`0x3D00`	`15616`
`0.5`	*E=62 (-1), M=0*	`0x3E00`	`15872`
`0.75`	*E=62 (-1), M=128*	`0x3E80`	`16000`
`1.0` (`F16One`)	*E=63 (0), M=0*	**`0x3F00`**	`16128`
`-1.0` (`F16NegOne`)	*S=1, E=63 (0), M=0*	**`0xBF00`**	`48896`
`1.4140625` (≈ √2)	*E=63 (0), M=106*	`0x3F6A`	`16234`
`1.5`	*E=63 (0), M=128*	`0x3F80`	`16256`
`2.0`	*E=64 (+1), M=0*	**`0x4000`**	`16384`
`-2.0`	*S=1, E=64 (+1), M=0*	`0xC000`	`49152`
`2.25`	*E=64 (+1), M=32*	`0x4020`	`16416`
`3.0`	*E=64 (+1), M=128*	`0x4080`	`16512`
`3.140625` (≈ π)	*E=64 (+1), M=146*	`0x4092`	`16530`
`4.0`	*E=65 (+2), M=0*	`0x4100`	`16640`
`10.0`	*E=66 (+3), M=64*	`0x4240`	`16960`
`100.0`	*E=69 (+6), M=144*	`0x4590`	`17808`
`256.0`	*E=71 (+8), M=0*	`0x4700`	`18176`

---

5. Calling-Convention & Gemeinsame Arbeitsvariablen der FPU

Alle FPU-Laufzeitroutinen teilen sich zur minimalen RAM-Belegung denselben Satz von **8 globalen 16-Bit Arbeitszellen** im Variablenbereich (ab Adresse `920` bei `SCREEN 32` bzw. `734` bei `SCREEN 64`):

Symbol im Compiler	Rolle in `__f16_add` / `__f16_sub`	Rolle in `__f16_mul`	Rolle in `__f16_div`	Rolle in `__f16_itof` / `__f16_ftoi`
`__r0`	**Eingabe Operand A**	**Eingabe A** / `fA_shift`	**Eingabe Dividend A**	—
`__r1`	**Eingabe Operand B**	**Eingabe B** / `fB_shift`	**Eingabe Divisor B**	—
`__fp_sign`	`signA` (`0x0000` / `0x8000`)	`signR` (`0x0000` / `0x8000`)	`signR` (`0x0000` / `0x8000`)	`sign` (`0x0000` / `0x8000`)
`__prod`	`signB` (`0x0000` / `0x8000`)	16-Bit `prod = fA * fB`	Schleifenzähler `cnt` (8..0)	—
`__quot`	Exponent `EA` / `ER`	Exponent `ER`	Exponent `ER`	Exponent `exp` (`E`)
`__rem`	Exponent `EB` / Differenz `dE`	—	Divisionsrest `rem`	—
`__f16_ma`	`|A|` → Mantisse `MA` / `MR`	Bruch `fA` → `MR`	Mantisse `MA` → `quot` (`MR`)	Arbeitswert `val` / `M`
`__f16_mb`	`|B|` → Mantisse `MB`	Bruch `fB`	Divisor-Mantisse `MB`	—

Übergabe- und Rückgabe-Konvention (ABI):

- **Binäre Routinen (`__f16_add` , `__f16_sub` , `__f16_mul` , `__f16_div`):**
- **Eingabe:** Linker Operand in `__r0` , rechter Operand in `__r1` .
- **Aufruf:** `CALL __f16_<op>`
- **Rückgabe:** Ergebnis im `Accu` (16-Bit `CLACK-F16`).
- **WICHTIG bei verschachtelten Ausdrücken (z. B. `A + (B / C)`):** Da `__f16_div` die Register `__r0` und `__r1` überschreibt, darf der linke Operand `A` noch **nicht** in `__r0` abgelegt werden, bevor `B / C` berechnet wird! Der Compiler speichert `A` stattdessen zuerst in einer temporären Zelle (`temp0` ab Adresse `952`), wertet `B / C` aus, speichert dessen `Accu` -Ergebnis nach `__r1` , lädt `temp0` nach `__r0` und ruft dann `CALL __f16_add` auf!
- **Unäre Konvertierungsroutinen (`__f16_itof` , `__f16_ftoi`):**
- **Eingabe:** Direkt im `Accu` .
- **Aufruf:** `CALL __f16_itof` bzw. `CALL __f16_ftoi`
- **Rückgabe:** Konvertiertes Ergebnis direkt im `Accu` .

---

6. Inline-Operationen & Compiler-Optimierungen für `CLACK-F16`

Der Compiler ruft für folgende Operationen **keine** Subroutine auf, sondern generiert extrem kompakten Inline-Code:

6.1 `F16ABS(x)` / `ABS(x#)` — Absolutbetrag (1 Befehl, 2 Worte!)

Löscht einfach Bit 15:

```
    ; Eingabe: CLACK-F16 Wert im Accu
    AND 0x7FFF            ; Vorzeichenbit löschen -> |x| im Accu
```

6.2 `F16NEG(x)` / `-x#` — Vorzeichenwechsel (2 Befehle, 4 Worte!)

Invertiert Bit 15 per `ADD 0x8000` , lässt aber `0.0` (`0x0000`) unverändert (damit kein `-0.0` = `0x8000` entsteht):

```
    ; Eingabe: CLACK-F16 Wert im Accu
    BZ __f16_neg_done     ; 0.0 bleibt kanonisch 0x0000!
    ADD 0x8000            ; Toggle Bit 15 (Vorzeichenwechsel)
__f16_neg_done:
```

6.3 Konvertierung `Q8.8` (`FIXED8`) → `CLACK-F16` (Inline nach `__f16_itof`)

Ein `Q8.8` -Wert W hat den realen Wert $\text{int16}(W) \cdot 2^{-8}$. Der Compiler konvertiert zuerst $\text{int16}(W)$ per `CALL __f16_itof` nach `CLACK-F16` und zieht anschließend (falls $\neq 0$) direkt `8` vom Exponenten (Bits 14..8) ab, indem er `0xF800` (`-8 << 8`) addiert:

```
    CALL __f16_itof       ; Konvertiere int16(W) -> CLACK-F16
    BZ __q88_to_f16_zero  ; 0 bleibt 0x0000
    ADD 0xF800            ; Exponent E = E - 8 (Teilen durch 256)
__q88_to_f16_zero:
```

6.4 Algebraische Konstanten-Optimierungen (Peephole)

- `X# + 0.0` oder `0.0 + X#` → Lädt nur `X#` (kein `CALL __f16_add`).
- `X# - 0.0` → Lädt nur `X#` .
- `X# 0.0` *oder* `0.0 X#` → `LDA 0x0000` (kein `CALL __f16_mul`).
- `X# 1.0` *(* `0x3F00` *)* *oder* `1.0 X#` → Lädt nur `X#` .
- `X# / 1.0` (`0x3F00`) → Lädt nur `X#` .
- `X# * X#` (Quadrat derselben Variable) → Lädt `X#` nur einmal und schreibt `STAM __r0` sowie direkt `STAM __r1` .

6.5 Gleitkomma-Vergleiche (`=` , `<>` , `<` , `>=` , `>` , `<=`) & `F16CMP(A, B)`

Alle Vergleiche auf `CLACK-F16` werden im Compiler über **`CALL __f16_sub`** realisiert:

- `__f16_sub(A, B)` liefert im `Accu` :
- `Accu == 0x0000` (`BZ` wahr) $\Longleftrightarrow A = B$
- `Bit 15 == 1` (`BN` wahr) $\Longleftrightarrow A < B$
- Abbildung aller 6 Vergleichsoperatoren:
- **`A == B`** : `CALL __f16_sub(A, B)` → `BZ true_label`
- **`A != B`** : `CALL __f16_sub(A, B)` → `BZ false_label` (invertiert)
- **`A < B`** : `CALL __f16_sub(A, B)` → `BN true_label`
- **`A >= B`** : `CALL __f16_sub(A, B)` → `BN false_label` (invertiert)
- **`A > B`** : Reihenfolge getauscht! `CALL __f16_sub(B, A)` → `BN true_label`
- **`A <= B`** : Reihenfolge getauscht! `CALL __f16_sub(B, A)` → `BN false_label` (invertiert)

---

7. Die 6 `CLACK-F16` Laufzeit-Subroutinen im Detail (Algorithmus & Exakter Assembler)

7.1 Gemeinsamer Normierungs-, Exponentenprüf- & Pack-Block (**`__f16_common_norm_pack`**)

Um kostbaren RAM zu sparen, teilen sich **`__f16_add`** , **`__f16_mul`** , **`__f16_div`** und **`__f16_itof`** denselben Ausgangsblock zum Normieren, Prüfen auf Underflow/Overflow und Zusammenpacken des 16-Bit `CLACK-F16` Wortes!

Vorbedingung beim Einsprung:

- `__fp_sign` enthält das Ziel-Vorzeichen (`0x0000` oder `0x8000`).
- `__quot` enthält den vorläufigen Exponenten `ER` .
- `__f16_ma` enthält die ungetrimmte Mantisse `MR` :
- Bei Einsprung in `__f16_common_norm_pack` : `MR` liegt im Bereich `[256 .. 1022]` . Falls Bit 9 (`0x0200`) gesetzt ist (`MR >= 512`), wird `MR >> 1` geschoben und `ER = ER + 1` erhöht.
- Bei Einsprung in `__f16_common_check_exp` : `MR` ist bereits auf `[256 .. 511]` normiert (`Bit 8 == 1`), es wird nur `ER` auf Underflow (`<= 0`) und Overflow (`>= 128`) geprüft.
- Bei Einsprung in `__f16_common_pack` : `ER` liegt sicher in `[1 .. 127]` und `MR` in `[256 .. 511]` .

Exakter Assembler-Code des gemeinsamen Pack-Blocks:

```
; ============================================================================
; GEMEINSAMER NORM-, EXPONENTEN- & PACK-BLOCK (fuer add, mul, div, itof)
; Erwartet: __fp_sign = signR (0x0000/0x8000), __quot = ER, __f16_ma = MR
; ============================================================================
__f16_common_norm_pack:
    LDAM __f16_ma               ; Lade MR fuer Bit-9 Check
    AND 0x0200                  ; Pruefe Carry in Bit 9 (MR >= 512)
    BZ __f16_common_check_exp   ; Wenn < 512 -> Direkt Exponent pruefen
    LDAM __f16_ma               ; Lade MR
    SHR                         ; MR >> 1
    STAM __f16_ma               ; Speichere normiertes MR
    LDAM __quot                 ; Lade ER
    ADD 1                       ; ER + 1
    STAM __quot                 ; Speichere ER

__f16_common_check_exp:
    LDAM __quot                 ; Lade ER
    BZ __f16_common_ret_zero    ; Wenn ER == 0 -> Underflow (Return 0.0)
    BN __f16_common_ret_zero    ; Wenn ER < 0  -> Underflow (Return 0.0)
    AND 0xFF80                  ; Pruefe ER >= 128 (Bits 15..7)
    BZ __f16_common_pack        ; Kein Overflow -> Packen

__f16_common_ovf:
    LDAM __fp_sign              ; Lade signR bei Overflow
    OR 0x7FFF                   ; Saettigung: signR | 0x7FFF (0x7FFF oder 0xFFFF)
    RET                         ; 1-Wort Opcode 0x6000

__f16_common_pack:
    LDAM __quot                 ; Lade ER (1..127)
    SHL                         ; ER << 1
    SHL                         ; ER << 2
    SHL                         ; ER << 3
    SHL                         ; ER << 4
    SHL                         ; ER << 5
    SHL                         ; ER << 6
    SHL                         ; ER << 7
    SHL                         ; ER << 8
    STAM __quot                 ; Speichere (ER << 8)
    LDAM __f16_ma               ; Lade MR
    AND 0x00FF                  ; Streife implizite 1 (Bit 8) ab: MR & 0x00FF
    STAM __f16_pack_m_op        ; Patche Mantisse in ADD-Operand (+5 Worte)
    LDAM __quot                 ; Lade (ER << 8)
    ADD 0x0000                  ; [Operand = __f16_pack_m_op] (ER << 8) | (MR & 0xFF)
    STAM __quot                 ; Speichere Betrag
    LDAM __fp_sign              ; Lade signR (0x0000 oder 0x8000)
    STAM __f16_pack_s_op        ; Patche signR in ADD-Operand (+5 Worte)
    LDAM __quot                 ; Lade Betrag
    ADD 0x0000                  ; [Operand = __f16_pack_s_op] signR | (ER << 8) | (MR & 0xFF)
    RET                         ; 1-Wort Opcode 0x6000

__f16_common_ret_zero:
    LDA 0x0000                  ; Kanonische Null 0.0
    RET                         ; 1-Wort Opcode 0x6000
```

7.2 Routine 1: `__f16_sub` (CLACK-F16 Subtraktion `__r0 - __r1`)

Funktionsweise:

1. Lädt Subtrahend B (`__r1`) und prüft `|B| = B & 0x7FFF == 0`. Falls $B = 0$, wird direkt A (`__r0`) im `Accu` per `RET` zurückgegeben.
2. Andernfalls wird das Vorzeichenbit von `__r1` per `ADD 0x8000` invertiert ($B \leftarrow -B$) und **per `BA __f16_add` (Tail-Call ohne `CALL`!)** direkt in die Additionsroutine gesprungen. Dadurch wird das 1-Level Return-Register `SR` geschont!

Exakter Assembler-Code (`__f16_sub`):

```
; ================================================================================
; SUBROUTINE: __f16_sub (__r0 - __r1 -> Accu)
; ================================================================================
__f16_sub:
    LDAM __r1                   ; Lade Subtrahend B
    AND 0x7FFF                  ; Pruefe |B| == 0
    BZ __f16_sub_ret_a          ; Wenn B == 0 -> Return A
    LDAM __r1                   ; Lade B fuer Vorzeichenwechsel
    ADD 0x8000                  ; B = -B (Toggle Bit 15)
    STAM __r1                   ; Speichere -B in __r1
    BA __f16_add                ; Tail-Jump in __f16_add (kein CALL!)
__f16_sub_ret_a:
    LDAM __r0                   ; Lade A unveraendert
    RET                         ; RETURN (B == 0)
```

7.3 Routine 2: `__f16_add` (CLACK-F16 Addition `__r0 + __r1`)

Mathematischer & Algorithmischer Ablauf im Compiler:

1. **Null-Prüfung:**

- Berechne $|A| = A \text{ AND } 0x7FFF$. Ist $|A| = 0$, gib B zurück.
- Berechne $|B| = B \text{ AND } 0x7FFF$. Ist $|B| = 0$, gib A zurück.

1. **Betrags-Sortierung ($|A| \geq |B|$ erzwingen):**

- Da in `CLACK-F16` der Exponent in Bits 14..8 und die Mantisse in Bits 7..0 liegt, ist der **unsigned Integer-Vergleich auf $|A|$ und $|B|$ (`0x0000..0x7FFF`) streng monoton zum reellen Absolutbetrag**!
- Berechne $|A| - |B|$ via `|A| + (~|B| + 1)`. Ist das Ergebnis negativ (`BN`), tausche `__r0` und `__r1`.
- Ab hier gilt garantiert: $|A| \geq |B|$ und damit $E_A \geq E_B$, und das Ergebnis hat (außer bei exakter Auslöschung zu `0.0`) **immer das Vorzeichen `signA` von A**!

1. **Entpacken:**

- `signA = A & 0x8000` → `__fp_sign`
- `signB = B & 0x8000` → `__prod`
- `EA = (A & 0x7F00) >> 8` → `__quot`
- `EB = (B & 0x7F00) >> 8` → `__rem`
- `MA = (A & 0x00FF) | 0x0100` → `__f16_ma`
- `MB = (B & 0x00FF) | 0x0100` → `__f16_mb`

1. **Exponenten-Differenz & Mantissen-Alignment:**

- `dE = EA - EB`. Wenn `dE == 0`, überspringe den Shift.
- Wenn `dE >= 9` (`dE + 0xFFF7` nicht negativ), fällt die 9-Bit Mantisse `MB` komplett aus der Genauigkeit → gib sofort A (`__r0`) zurück!
- Sonst schiebe `MB` in einer Schleife genau `dE`-mal per `SHR` um 1 Bit nach rechts.

1. **Addition vs. Subtraktion der Mantissen:**

- Prüfe `(signA + signB) == 0`:
- **Gleiche Vorzeichen (`signA == signB`):** `MR = MA + MB` → springe zu `__f16_common_norm_pack` (prüft Bit 9 `0x0200` und packt).
- **Unterschiedliche Vorzeichen (`signA != signB`):** `MR = MA - MB`.
- Ist `MR == 0` (exakte Auslöschung $x + (-x)$) → springe zu `__f16_common_ret_zero` (`0x0000`).
- **Links-Normierungsschleife:** Solange Bit 8 (`MR & 0x0100`) gleich `0` ist: dekrementiere `EA = EA - 1` (falls `EA == 0` → Underflow `0x0000`) und schiebe `MR = MR << 1` (`SHL`). Danach Sprung zu `__f16_common_pack`.

Exakter Assembler-Code (`__f16_add`):

```
; ============================================================================
; SUBROUTINE: __f16_add (__r0 + __r1 -> Accu)
; ============================================================================
__f16_add:
    LDAM __r0                   ; Lade Operand A
    AND 0x7FFF                  ; |A| = A & 0x7FFF
    BZ __f16_add_ret_b          ; Wenn |A| == 0 -> Return B
    STAM __f16_ma               ; Speichere |A|

    LDAM __r1                   ; Lade Operand B
    AND 0x7FFF                  ; |B| = B & 0x7FFF
    BZ __f16_add_ret_a          ; Wenn |B| == 0 -> Return A
    STAM __f16_mb               ; Speichere |B|

    ; Vergleiche |A| und |B|: wenn |A| < |B|, tausche __r0 und __r1
    NOT                         ; ~|B|
    ADD 1                       ; -|B|
    STAM __f16_add_patch_mag    ; Patche -|B| in ADD (+5 Worte)
    LDAM __f16_ma               ; Lade |A|
    ADD 0x0000                  ; [Operand = __f16_add_patch_mag] |A| - |B|
    BN __f16_add_swap           ; Wenn |A| < |B| -> Swap A und B
    BA __f16_add_unpack

__f16_add_swap:
    LDAM __r0
    STAM __f16_ma               ; Temp = A
    LDAM __r1
    STAM __r0                   ; __r0 = B
    LDAM __f16_ma
    STAM __r1                   ; __r1 = A

__f16_add_unpack:
    LDAM __r0
    AND 0x8000
    STAM __fp_sign              ; signA = A & 0x8000

    LDAM __r1
    AND 0x8000
    STAM __prod                 ; signB = B & 0x8000

    LDAM __r0
    AND 0x7F00
    SHR                         ; EA >> 1 .. >> 8 (8x SHR)
    SHR
    SHR
    SHR
    SHR
    SHR
    SHR
    SHR
    STAM __quot                 ; EA in __quot

    LDAM __r1
    AND 0x7F00
    SHR                         ; EB >> 1 .. >> 8 (8x SHR)
    SHR
    SHR
    SHR
    SHR
    SHR
    SHR
    SHR
    STAM __rem                  ; EB in __rem

    LDAM __r0
    AND 0x00FF
    OR 0x0100                   ; MA = (A & 0xFF) | 0x0100
    STAM __f16_ma

    LDAM __r1
    AND 0x00FF
```

```
    OR 0x0100                   ; MB = (B & 0xFF) | 0x0100
    STAM __f16_mb

    ; Berechne dE = EA - EB (>= 0 da |A| >= |B|)
    LDAM __rem                  ; Lade EB
    NOT                         ; ~EB
    ADD 1                       ; -EB
    STAM __f16_add_patch_de     ; Patche -EB in ADD (+5 Worte)
    LDAM __quot                 ; Lade EA
    ADD 0x0000                  ; [Operand = __f16_add_patch_de] dE = EA - EB
    BZ __f16_add_aligned        ; Wenn dE == 0 -> Keine Ausrichtung noetig
    STAM __rem                  ; Speichere dE

    ADD 0xFFF7                  ; Pruefe dE - 9
    BN __f16_add_shift_loop     ; Wenn dE < 9 -> Mantissen-Shift-Schleife
    BA __f16_add_ret_a          ; Wenn dE >= 9 -> B faellt aus 9-Bit Mantisse -> Return A

__f16_add_shift_loop:
    LDAM __f16_mb               ; Lade MB
    SHR                         ; MB >> 1
    STAM __f16_mb               ; Speichere MB
    LDAM __rem                  ; Lade dE
    ADD 0xFFFF                  ; dE - 1
    STAM __rem                  ; Speichere dE
    BZ __f16_add_aligned        ; Wenn dE == 0 -> Ausrichtung fertig
    BA __f16_add_shift_loop

__f16_add_aligned:
    LDAM __prod                 ; Lade signB
    STAM __f16_add_patch_sign   ; Patche signB in ADD (+5 Worte)
    LDAM __fp_sign              ; Lade signA
    ADD 0x0000                  ; [Operand = __f16_add_patch_sign] signA + signB (XOR auf Bit 15)
    BZ __f16_add_same_sign      ; Wenn gleiche Vorzeichen -> Addiere Mantissen
    BA __f16_add_diff_sign      ; Unterschiedliche Vorzeichen -> Subtrahiere Mantissen

__f16_add_same_sign:
    LDAM __f16_mb               ; Lade MB
    STAM __f16_add_patch_addm   ; Patche +MB in ADD (+5 Worte)
    LDAM __f16_ma               ; Lade MA
    ADD 0x0000                  ; [Operand = __f16_add_patch_addm] MR = MA + MB
    STAM __f16_ma               ; Speichere MR
    BA __f16_common_norm_pack   ; Gemeinsamer Norm+Pack Block

__f16_add_diff_sign:
    LDAM __f16_mb               ; Lade MB
    NOT                         ; ~MB
    ADD 1                       ; -MB
    STAM __f16_add_patch_subm   ; Patche -MB in ADD (+5 Worte)
    LDAM __f16_ma               ; Lade MA
    ADD 0x0000                  ; [Operand = __f16_add_patch_subm] MR = MA - MB
    BZ __f16_common_ret_zero    ; Wenn MR == 0 -> Exakte Ausloeschung -> Return 0.0
    STAM __f16_ma               ; Speichere MR

__f16_add_norm_loop:
    LDAM __f16_ma               ; Lade MR fuer Normierung
    AND 0x0100                  ; Pruefe implizite 1 (Bit 8)
    BZ __f16_add_norm_shift     ; Wenn Bit 8 == 0 -> Links-Shift
    BA __f16_common_pack        ; Wenn Bit 8 == 1 -> Direkt Packen (EA ist bereits >= 1)

__f16_add_norm_shift:
    LDAM __quot                 ; Lade EA
    ADD 0xFFFF                  ; EA - 1
    BZ __f16_common_ret_zero    ; Wenn EA == 0 -> Underflow 0.0
    STAM __quot                 ; Speichere EA
    LDAM __f16_ma               ; Lade MR
    SHL                         ; MR << 1
    STAM __f16_ma               ; Speichere MR
    BA __f16_add_norm_loop

__f16_add_ret_a:
    LDAM __r0
```

```
    RET
__f16_add_ret_b:
    LDAM __r1
    RET
```

7.4 Routine 3: `__f16_mul` (CLACK-F16 Multiplikation `__r0 * __r1` ohne 32-Bit-Überlauf!)

Der geniale 16-Bit-Zerlegungstrick des Compilers:

Würde man die zwei 9-Bit Mantissen $M_A = 256 + f_A$ und $M_B = 256 + f_B$ ($f_A, f_B \in [0..255]$) direkt multiplizieren, wäre das maximale Produkt $511 \times 511 = 261\,121 > 65\,535$ und würde ein 16-Bit-Register sprengen!

Deshalb zerlegt unser Compiler das Produkt algebraisch exakt:

$$M_R = \frac{(256 + f_A) \cdot (256 + f_B)}{256} = 256 + f_A + f_B + \left\lfloor \frac{f_A \cdot f_B}{256} \right\rfloor = \text{0x0100} + f_A + f_B + \big((f_A \cdot f_B) \gg 8\big)$$

Da $f_A, f_B \le 255$, ist $f_A \cdot f_B \le 255 \times 255 = 65\,025 \le 65\,535$!

Das 8×8-Bit Produkt `prod = fA * fB` passt damit zu 100 % überlauffrei in ein einziges 16-Bit-Wort!

Ablauf:

1. Wenn $|A| == 0$ oder $|B| == 0$ → Return `0x0000`.
2. `signR = (A & 0x8000) ^ (B & 0x8000)` in `__fp_sign`.
3. `ER = EA + EB - 63` (`ADD 0xFFC1`) in `__quot`.
4. Extrahiere $f_A = A \text{ AND } \text{0x00FF}$ und $f_B = B \text{ AND } \text{0x00FF}$.
5. Berechne `prod = fA * fB` in max. 8 Shift-and-Add-Schritten (nutzt `BE` für `LSB == 0`).
6. Berechne `MR = 0x0100 + (prod >> 8) + fA + fB` in `__f16_ma`.
7. Springe zu `__f16_common_norm_pack` (korrigiert `MR >= 512` per `SHR` & `ER++`, prüft Underflow/Overflow und packt).

Exakter Assembler-Code (`__f16_mul`):

```
; ============================================================================
; SUBROUTINE: __f16_mul (__r0 * __r1 -> Accu)
; ============================================================================
__f16_mul:
    LDAM __r0
    AND 0x7FFF
    BZ __f16_common_ret_zero    ; Wenn |A| == 0 -> Return 0.0

    LDAM __r1
    AND 0x7FFF
    BZ __f16_common_ret_zero    ; Wenn |B| == 0 -> Return 0.0

    ; Berechne signR = (A ^ B) & 0x8000
    LDAM __r0
    AND 0x8000
    STAM __fp_sign              ; signR = signA
    LDAM __r1
    BN __f16_mul_b_neg          ; Wenn B < 0 (Bit 15 == 1) -> Toggle signR
    BA __f16_mul_exp

__f16_mul_b_neg:
    LDAM __fp_sign
    ADD 0x8000                  ; signR ^= 0x8000
    STAM __fp_sign

__f16_mul_exp:
    LDAM __r0
    AND 0x7F00
    SHR                         ; EA >> 8 (8x SHR)
    SHR
    SHR
    SHR
    SHR
    SHR
    SHR
    SHR
    STAM __quot                 ; Speichere EA

    LDAM __r1
    AND 0x7F00
    SHR                         ; EB >> 8 (8x SHR)
    SHR
    SHR
    SHR
    SHR
    SHR
    SHR
    SHR
    STAM __f16_mul_patch_exp    ; Patche +EB in ADD (+5 Worte)
    LDAM __quot                 ; Lade EA
    ADD 0x0000                  ; [Operand = __f16_mul_patch_exp] EA + EB
    ADD 0xFFC1                  ; ER = EA + EB - 63 (Bias)
    STAM __quot                 ; Speichere ER

    ; Extrahiere Bruch-Mantissen fA = A & 0xFF und fB = B & 0xFF
    LDAM __r0
    AND 0x00FF
    STAM __f16_ma               ; fA sichern
    STAM __r0                   ; fA_shift = fA

    LDAM __r1
    AND 0x00FF
    STAM __f16_mb               ; fB sichern
    STAM __r1                   ; fB_shift = fB

    ; 8x8-Bit Shift-and-Add Multiplikation: prod = fA * fB (max 65025)
    LDA 0
    STAM __prod                 ; prod = 0
    LDAM __r1                   ; Lade fB_shift
    BZ __f16_mul_combine        ; Wenn fB == 0 -> Skip Loop
```

```
__f16_mul_loop:
    BE __f16_mul_skip_add       ; Wenn LSB(fB_shift) == 0 -> Skip Add
    LDAM __r0                   ; Lade fA_shift
    STAM __f16_mul_patch_add    ; Patche +fA_shift in ADD (+5 Worte)
    LDAM __prod                 ; Lade prod
    ADD 0x0000                  ; [Operand = __f16_mul_patch_add] prod + fA_shift
    STAM __prod                 ; Speichere prod

__f16_mul_skip_add:
    LDAM __r0
    SHL                         ; fA_shift << 1
    STAM __r0
    LDAM __r1
    SHR                         ; fB_shift >> 1
    BZ __f16_mul_combine        ; Wenn fB_shift == 0 -> Fertig!
    STAM __r1
    BA __f16_mul_loop

__f16_mul_combine:
    LDAM __prod                 ; Lade prod
    SHR                         ; prod >> 8 (8x SHR)
    SHR
    SHR
    SHR
    SHR
    SHR
    SHR
    SHR
    ADD 0x0100                  ; 256 + (prod >> 8)
    STAM __prod

    LDAM __f16_ma               ; Lade fA
    STAM __f16_mul_patch_fa     ; Patche +fA in ADD (+5 Worte)
    LDAM __prod
    ADD 0x0000                  ; [Operand = __f16_mul_patch_fa] + fA
    STAM __prod

    LDAM __f16_mb               ; Lade fB
    STAM __f16_mul_patch_fb     ; Patche +fB in ADD (+5 Worte)
    LDAM __prod
    ADD 0x0000                  ; [Operand = __f16_mul_patch_fb] MR = 256 + fA + fB + (prod >> 8)
    STAM __f16_ma               ; Speichere MR in __f16_ma
    BA __f16_common_norm_pack   ; Weiter zu gemeinsamem Norm+Pack Block
```

7.5 Routine 4: `__f16_div` (CLACK-F16 Division `__r0 / __r1` in 8 Restoring-Schritten)

Funktionsweise:

1. Berechne `signR = (A ^ B) & 0x8000` in `__fp_sign`.
2. Wenn $|A| == 0 \rightarrow$ Return `0x0000`. Wenn $|B| == 0 \rightarrow$ Sättigung `__f16_common_ovf` (`signR | 0x7FFF`).
3. Berechne vorläufigen Exponenten `ER = EA - EB + 63` in `__quot`.
4. Entpacke $M_A = (A \text{ AND } \text{0xFF}) \mid \text{0x0100}$ und $M_B = (B \text{ AND } \text{0xFF}) \mid \text{0x0100}$.
5. Berechne einmalig `-MB = ~MB + 1` und **patche damit vorab gleich alle 3 Subtraktionsstellen** in der Routine!
6. **Vorab-Normierung des Restes (`rem`):**

- Falls $M_A \geq M_B$: Die führende `1` des Quotienten (`quot = 1`) passt direkt; der Startrest ist `rem = MA - MB`.
- Falls $M_A < M_B$: Schiebe M_A einmal nach links (`2 MA`), *setze* `rem = 2 MA - MB` und dekrementiere `ER = ER - 1`.
- Damit steht das führende Bit (`quot = 1`) bereits fest und es gilt garantiert $0 \leq \text{rem} < M_B$!

1. **Exakt 8 Schleifendurchläufe (`cnt = 8`):**

- `quot = quot << 1`

- `rem = rem << 1`
- Prüfe `(rem << 1) - MB`: Falls nicht negativ (`BN` springt nicht), setze `rem = (rem << 1) - MB` und `quot = quot | 1`.

1. Nach 8 Schritten liegt `quot` normiert in `[256 .. 511]` in `__f16_ma` → Sprung zu `__f16_common_check_exp`.

Exakter Assembler-Code (`__f16_div`):

```
; ================================================================================
; SUBROUTINE: __f16_div (__r0 / __r1 -> Accu)
; ================================================================================
__f16_div:
    LDAM __r0
    AND 0x8000
    STAM __fp_sign              ; signR = signA
    LDAM __r1
    BN __f16_div_b_neg          ; Wenn B < 0 -> Toggle signR
    BA __f16_div_check_zero

__f16_div_b_neg:
    LDAM __fp_sign
    ADD 0x8000
    STAM __fp_sign

__f16_div_check_zero:
    LDAM __r0
    AND 0x7FFF
    BZ __f16_common_ret_zero    ; 0.0 / B = 0.0

    LDAM __r1
    AND 0x7FFF
    BZ __f16_common_ovf         ; A / 0.0 = Saettigung (signR | 0x7FFF)

    ; Berechne ER = EA - EB + 63
    LDAM __r0
    AND 0x7F00
    SHR                         ; EA >> 8 (8x SHR)
    SHR
    SHR
    SHR
    SHR
    SHR
    SHR
    SHR
    STAM __quot                 ; EA

    LDAM __r1
    AND 0x7F00
    SHR                         ; EB >> 8 (8x SHR)
    SHR
    SHR
    SHR
    SHR
    SHR
    SHR
    SHR
    NOT                         ; ~EB
    ADD 1                       ; -EB
    STAM __f16_div_patch_eb     ; Patche -EB in ADD (+5 Worte)
    LDAM __quot                 ; Lade EA
    ADD 0x0000                  ; [Operand = __f16_div_patch_eb] EA - EB
    ADD 63                      ; ER = EA - EB + 63
    STAM __quot                 ; Speichere ER

    ; Entpacke MA und MB (jeweils 9 Bit: 256..511)
    LDAM __r0
    AND 0x00FF
    OR 0x0100
    STAM __f16_ma               ; MA

    LDAM __r1
    AND 0x00FF
    OR 0x0100
    STAM __f16_mb               ; MB

    ; Berechne -MB einmalig und patche alle 3 Subtraktions-Operanden
    NOT                         ; ~MB
    ADD 1                       ; -MB
    STAM __f16_div_sub_init     ; Patch 1: MA - MB
```

```
    STAM __f16_div_sub_lt       ; Patch 2: 2*MA - MB
    STAM __f16_div_sub_loop     ; Patch 3: (rem << 1) - MB

    ; Pruefe ob MA < MB
    LDAM __f16_ma
    ADD 0x0000                  ; [Operand = __f16_div_sub_init] MA - MB
    BN __f16_div_ma_lt_mb       ; Wenn MA < MB -> Vorab-Shift 2*MA und ER--
    STAM __rem                  ; rem = MA - MB
    BA __f16_div_loop_init

__f16_div_ma_lt_mb:
    LDAM __f16_ma
    SHL                         ; 2 * MA
    ADD 0x0000                  ; [Operand = __f16_div_sub_lt] 2*MA - MB
    STAM __rem                  ; rem = 2*MA - MB
    LDAM __quot
    ADD 0xFFFF                  ; ER = ER - 1
    STAM __quot

__f16_div_loop_init:
    LDA 1
    STAM __f16_ma               ; quot = 1 (fuehrende 1)
    LDA 8
    STAM __prod                 ; cnt = 8 Nachkommabits

__f16_div_loop:
    LDAM __f16_ma
    SHL                         ; quot << 1
    STAM __f16_ma

    LDAM __rem
    SHL                         ; rem << 1
    STAM __rem
    ADD 0x0000                  ; [Operand = __f16_div_sub_loop] (rem << 1) - MB
    BN __f16_div_next           ; Wenn (rem << 1) < MB -> Bit bleibt 0

    STAM __rem                  ; rem = (rem << 1) - MB
    LDAM __f16_ma
    OR 1                        ; quot |= 1
    STAM __f16_ma

__f16_div_next:
    LDAM __prod
    ADD 0xFFFF                  ; cnt - 1
    STAM __prod
    BZ __f16_common_check_exp   ; Wenn 8 Bits fertig -> Pruefe Exponent & Packen
    BA __f16_div_loop
```

7.6 Routine 5: `__f16_itof` (16-Bit Signed Integer `Accu` → `CLACK-F16` `Accu`)

Funktionsweise:

- Ist `Accu == 0` → Return `0x0000`.
- Bestimmt Vorzeichen `sign` (`0x0000` oder `0x8000`) und bildet den 16-Bit Absolutbetrag `|val|`.
- Setzt den Start-Exponenten auf **`exp = 71`** (**`63 + 8`**), denn wenn das führende Bit bereits auf Bit 8 (`0x0100`, Wert `256`) liegt, ist $256 = 1.0 \cdot 2^8 \Longrightarrow E = 63 + 8 = 71$.
- **Falls `|val| >= 512`** (**`|val| & 0xFE00 != 0`**): Schiebe `|val|` in einer Schleife per `SHR` nach rechts und inkrementiere `exp++`, bis alle Bits oberhalb von Bit 8 Null sind (dabei wird z. B. auch `-32768` = `0x8000` sauber als `|val| = 32768` → 7× `SHR` → `exp = 78`, `M = 0` → `0xCE00` konvertiert!).
- **Falls `|val| < 256`** (**`|val| & 0x0100 == 0`**): Schiebe `|val|` in einer Schleife per `SHL` nach links und dekrementiere `exp--`, bis Bit 8 (`0x0100`) gesetzt ist.
- Springe zu `__f16_common_pack`.

Exakter Assembler-Code (`__f16_itof`):

```
; ================================================================================
; SUBROUTINE: __f16_itof (Accu Int16 -> Accu CLACK-F16)
; ================================================================================
__f16_itof:
    BZ __f16_common_ret_zero    ; Wenn 0 -> Return 0.0
    STAM __f16_ma               ; Speichere Eingabe
    LDA 0
    STAM __fp_sign              ; sign = 0
    LDAM __f16_ma
    BN __f16_itof_neg           ; Wenn negativ -> Negiere
    BA __f16_itof_norm

__f16_itof_neg:
    NOT                         ; ~val
    ADD 1                       ; |val| = -val
    STAM __f16_ma
    LDA 0x8000
    STAM __fp_sign              ; sign = 0x8000

__f16_itof_norm:
    LDA 71                      ; Init exp = 71 (Bias 63 + 8)
    STAM __quot

__f16_itof_shr_check:
    LDAM __f16_ma
    AND 0xFE00                  ; Pruefe ob |val| >= 512 (Bits 15..9 gesetzt)
    BZ __f16_itof_shl_check     ; Wenn < 512 -> Pruefe Links-Shift
    LDAM __f16_ma
    SHR                         ; |val| >> 1
    STAM __f16_ma
    LDAM __quot
    ADD 1                       ; exp + 1
    STAM __quot
    BA __f16_itof_shr_check

__f16_itof_shl_check:
    LDAM __f16_ma
    AND 0x0100                  ; Pruefe Bit 8 (0x0100)
    BZ __f16_itof_shl_step      ; Wenn Bit 8 == 0 -> Links-Shift
    BA __f16_common_pack        ; Wenn Bit 8 == 1 -> Packen!

__f16_itof_shl_step:
    LDAM __f16_ma
    SHL                         ; |val| << 1
    STAM __f16_ma
    LDAM __quot
    ADD 0xFFFF                  ; exp - 1
    STAM __quot
    BA __f16_itof_shl_check
```

7.7 Routine 6: `__f16_ftoi` (`CLACK-F16` `Accu` → 16-Bit Signed Integer `Accu`)

Funktionsweise (Truncation Richtung Null mit Sättigung):

- Extrahiere `sign = Wort & 0x8000` und `E = (Wort & 0x7F00) >> 8`.
- **Wenn `E < 63` (`E + 0xFFC1` negativ):** $|x| < 1.0$ → Return `0`.
- **Wenn `E >= 78` (`E + 0xFFB2` nicht negativ):** $|x| \geq 32768$ → Sättigung auf `0x7FFF` (`+32767`) bei `sign == 0` bzw. `0x8000` (`-32768`) bei `sign != 0`.
- Entpacke `M = (Wort & 0x00FF) | 0x0100`.
- Vergleiche `E` mit `71` (`E - 71` via `ADD 0xFFB9`):
- `E == 71`: `M` hat bereits die richtige Größenordnung (`256..511`).
- `E > 71`: Schiebe `M` genau `(E - 71)`-mal per `SHL` nach links.
- `E < 71`: Schiebe `M` genau `(71 - E)`-mal per `SHR` nach rechts.
- Wende bei `sign != 0` das Zweierkomplement (`~M + 1`) an und kehre per `RET` zurück.

Exakter Assembler-Code (`__fl6_ftoi`):

```
; ================================================================================
; SUBROUTINE: __f16_ftoi (Accu CLACK-F16 -> Accu Int16)
; ================================================================================
__f16_ftoi:
    STAM __f16_ma               ; Speichere F16-Wort
    AND 0x8000
    STAM __fp_sign              ; Speichere sign

    LDAM __f16_ma
    AND 0x7F00
    SHR                         ; E >> 8 (8x SHR)
    SHR
    SHR
    SHR
    SHR
    SHR
    SHR
    SHR
    STAM __quot                 ; Speichere E

    ADD 0xFFC1                  ; Pruefe E - 63
    BN __f16_ftoi_ret_zero      ; Wenn E < 63 (|x| < 1.0) -> Return 0

    LDAM __quot
    ADD 0xFFB2                  ; Pruefe E - 78
    BN __f16_ftoi_extract_m     ; Wenn E < 78 -> Normale Konvertierung
    LDAM __fp_sign              ; Overflow: Pruefe Sign
    BZ __f16_ftoi_ovf_pos       ; Positiv -> +32767
    LDA 0x8000                  ; Negativ -> -32768
    RET

__f16_ftoi_ovf_pos:
    LDA 0x7FFF                  ; +32767
    RET

__f16_ftoi_extract_m:
    LDAM __f16_ma
    AND 0x00FF
    OR 0x0100                   ; M = (Wort & 0xFF) | 0x0100
    STAM __f16_ma

    LDAM __quot
    ADD 0xFFB9                  ; Berechne E - 71
    BZ __f16_ftoi_apply_sign    ; Wenn E == 71 -> Kein Shift noetig
    BN __f16_ftoi_shr_init      ; Wenn E < 71  -> Rechts-Shift

    ; E > 71: Links-Shift um cnt = (E - 71)
    STAM __quot                 ; cnt = E - 71
__f16_ftoi_shl_loop:
    LDAM __f16_ma
    SHL                         ; M << 1
    STAM __f16_ma
    LDAM __quot
    ADD 0xFFFF                  ; cnt - 1
    STAM __quot
    BZ __f16_ftoi_apply_sign
    BA __f16_ftoi_shl_loop

__f16_ftoi_shr_init:
    NOT                         ; ~(E - 71)
    ADD 1                       ; cnt = 71 - E
    STAM __quot
__f16_ftoi_shr_loop:
    LDAM __f16_ma
    SHR                         ; M >> 1
    STAM __f16_ma
    LDAM __quot
    ADD 0xFFFF                  ; cnt - 1
    STAM __quot
    BZ __f16_ftoi_apply_sign
    BA __f16_ftoi_shr_loop
```

```
__f16_ftoi_apply_sign:
    LDAM __fp_sign
    BZ __f16_ftoi_done          ; Wenn positiv -> Fertig
    LDAM __f16_ma
    NOT
    ADD 1                       ; -Betrag
    RET

__f16_ftoi_done:
    LDAM __f16_ma
    RET

__f16_ftoi_ret_zero:
    LDA 0
    RET
```

---

8. Format 2: Schnelles 16-Bit Festkomma `Q8.8` (`FIXED8`)

Neben `CLACK-F16` unterstützt der Compiler (`pkg/codegen/fixed_q88.go`) das vorzeichenbehaftete 16-Bit Festkommaformat **`Q8.8` (`FIXED8`)**, das für Echtzeit-Grafik, Harmonographen, Sinus/Cosinus und Mandelbrot-Fraktale auf dem Relaisrechner optimal ist.

8.1 Datenformat & Wertebereich (`Q8.8`)

- **Darstellung:** Vorzeichenbehaftetes 16-Bit Zweierkomplement (`int16`), skaliert mit Faktor `256` (`2^8`):

$$\text{Wert} = \frac{\text{int16}(W)}{256.0}, \quad W = \text{round}(\text{Wert} \times 256)$$

- **Bits 15..8:** Vorzeichenbehafteter Vorkommaanteil (`-128 .. +127`).
- **Bits 7..0:** 8-Bit Nachkommaanteil (Auflösung `1/256 = 0.00390625`).
- **Wertebereich:** `-128.0000` (`0x8000`) bis `+127.99609375` (`0x7FFF`).
- **Typische Konstanten:** `1.0` = `0x0100` (`256`), `0.5` = `0x0080` (`128`), `-1.0` = `0xFF00` (`65280`), `2.5` = `0x0280` (`640`).

8.2 Inline-Operationen in `Q8.8` (`+`, `-`, `ITOF`, Vergleiche)

1. **Addition (`A + B`) und Subtraktion (`A - B`):**

- **100 % identisch zur normalen 16-Bit Integer-Addition/Subtraktion!** Kein Unterprogramm nötig!
- Konstante rechts: `LDAM A` → `ADD #const_q88` (nur 2 Befehle!).
- Variable rechts: Nutzt exakt **Idiom 1 (`ADD`-Patch)** bzw. **Idiom 2 (`SUB`-Patch)**.

1. **Konvertierung `INT16` → `Q8.8` (`ITOF(n)`):**

- Multiplikation mit `256` durch **8 direkte `SHL`-Befehle inline** (`Accu << 8`).

1. **Vergleiche (`=`, `<>`, `<`, `<=`, `>`, `>=`):**

- Identisch zum 16-Bit Zweierkomplement-Vergleich (`A - B` berechnen und mit `BZ` / `BN` verzweigen).

8.3 Die 3 `Q8.8` Laufzeit-Subroutinen (`__runtime_fmul_q88`, `__runtime_fdiv_q88`, `__runtime_ftoi_q88`)

A. `__runtime_fmul_q88` — 32-Bit Zwischenprodukt `(|A| * |B|) >> 8` mit Sättigung

Da zwei 16-Bit `Q8.8`-Zahlen beim Multiplizieren bis zu 30 Bit benötigen, bildet `__runtime_fmul_q88` zuerst die Beträge $|A|$ und $|B|$ sowie das Zielvorzeichen `sign` (`0` oder `1` in `__fp_sign`) und führt eine **32-Bit Shift-and-Add Multiplikation** mit `a_hi:a_lo` (`__quot:__r0`) und `prod_hi:prod_lo` (`__rem:__prod`) durch:

- Nach der Schleife wird geprüft, ob `prod_hi >= 128` (`prod_hi & 0xFF80 != 0`): Falls ja, erfolgt **Overflow-Sättigung** auf `0x7FFF` (`+127.996`) bzw. `0x8000` (`-128.0`).

- Andernfalls ist das skalierte 16-Bit-Ergebnis exakt `(prod_hi << 8) + (prod_lo >> 8)`, worauf bei `sign == 1` das Zweierkomplement angewandt wird.

```
; ================================================================================
; SUBROUTINE: __runtime_fmul_q88 (__r0 * __r1 >> 8 -> Accu)
; ================================================================================
__runtime_fmul_q88:
    LDA 0
    STAM __prod                 ; prod_lo = 0
    STAM __rem                  ; prod_hi = 0
    STAM __quot                 ; a_hi = 0
    STAM __fp_sign              ; sign = 0

    LDAM __r0                   ; Lade A
    BZ __fmul_q88_ret_zero      ; A == 0 -> Return 0
    BN __fmul_q88_a_neg         ; A < 0  -> Negiere A
    BA __fmul_q88_check_b

__fmul_q88_a_neg:
    NOT
    ADD 1
    STAM __r0                   ; |A| = -A
    LDA 1
    STAM __fp_sign              ; sign = 1

__fmul_q88_check_b:
    LDAM __r1                   ; Lade B
    BZ __fmul_q88_ret_zero      ; B == 0 -> Return 0
    BN __fmul_q88_b_neg         ; B < 0  -> Negiere B
    BA __fmul_q88_loop

__fmul_q88_b_neg:
    NOT
    ADD 1
    STAM __r1                   ; |B| = -B
    LDAM __fp_sign
    ADD 1
    AND 1                       ; sign = sign XOR 1
    STAM __fp_sign
    LDAM __r1                   ; Lade |B| fuer Loop-Start

__fmul_q88_loop:
    BE __fmul_q88_skip_add      ; Wenn LSB(B) == 0 -> Skip Add
    LDAM __r0                   ; Lade a_lo
    STAM __fmul_q88_patch_lo    ; Patche ADD a_lo (+5 Worte)
    LDAM __prod                 ; Lade prod_lo
    ADD 0x0000                  ; [Operand = __fmul_q88_patch_lo] prod_lo + a_lo
    STAM __prod
    BC __fmul_q88_carry         ; Wenn 16-Bit Carry -> prod_hi++
    BA __fmul_q88_add_hi

__fmul_q88_carry:
    LDAM __rem
    ADD 1                       ; prod_hi + 1
    STAM __rem

__fmul_q88_add_hi:
    LDAM __quot                 ; Lade a_hi
    STAM __fmul_q88_patch_hi    ; Patche ADD a_hi (+5 Worte)
    LDAM __rem                  ; Lade prod_hi
    ADD 0x0000                  ; [Operand = __fmul_q88_patch_hi] prod_hi + a_hi
    STAM __rem

__fmul_q88_skip_add:
    LDAM __r1
    SHR                         ; B >> 1
    BZ __fmul_q88_scale         ; Wenn B == 0 -> Skalierung >> 8
    STAM __r1

    ; 32-Bit Links-Shift (a_hi:a_lo) << 1
    LDAM __quot
    SHL                         ; a_hi << 1
    STAM __quot
    LDAM __r0
```

```
    SHL                         ; a_lo << 1
    STAM __r0
    BC __fmul_q88_shl_carry     ; Carry aus a_lo in Bit 0 von a_hi uebertragen
    LDAM __r1
    BA __fmul_q88_loop

__fmul_q88_shl_carry:
    LDAM __quot
    OR 1
    STAM __quot
    LDAM __r1
    BA __fmul_q88_loop

__fmul_q88_scale:
    LDAM __rem                  ; Lade prod_hi
    AND 0xFF80                  ; Pruefe Overflow (prod_hi >= 128)
    BZ __fmul_q88_no_ovf
    LDAM __fp_sign
    BZ __fmul_q88_ovf_pos
    LDA 0x8000                  ; Overflow negativ -> -128.0
    RET
__fmul_q88_ovf_pos:
    LDA 0x7FFF                  ; Overflow positiv -> +127.996
    RET

__fmul_q88_no_ovf:
    LDAM __rem
    SHL                         ; prod_hi << 8 (8x SHL)
    SHL
    SHL
    SHL
    SHL
    SHL
    SHL
    SHL
    STAM __rem
    LDAM __prod
    SHR                         ; prod_lo >> 8 (8x SHR)
    SHR
    SHR
    SHR
    SHR
    SHR
    SHR
    SHR
    STAM __fmul_q88_patch_comb  ; Patche (prod_lo >> 8) in ADD (+5 Worte)
    LDAM __rem
    ADD 0x0000                  ; [Operand = __fmul_q88_patch_comb] (prod_hi << 8) + (prod_lo >> 8)
    STAM __prod

    LDAM __fp_sign
    BZ __fmul_q88_done
    LDAM __prod
    NOT
    ADD 1
    RET
__fmul_q88_done:
    LDAM __prod
    RET
__fmul_q88_ret_zero:
    LDA 0
    RET
```

B. `__runtime_fdiv_q88` — 24-Bit / 16-Bit Restoring-Division `(|A| << 8) / |B|`

Teilt den 24-Bit Dividenden `|A| << 8` auf in das obere Wort `rem = |A| >> 8` (`__rem`) und das untere Wort `num_lo = (|A| & 0xFF) << 8` (`__r0`). Prüft vorab `rem >= |B|` (Overflow) und schiebt in 16 Iterationen (`cnt = 16` in `__prod`) jeweils das höchste Bit von `num_lo` per `SHL` + `BC` in `rem` hinein.

C. `__runtime_ftoi_q88` — Vorzeichenrichtige Division durch 256 Richtung Null (`Q8.8` → `INT16`)

Da ein einfacher Rechtsshift bei negativen Zweierkomplementzahlen abrunden würde (bzw. logisch `0` nachschiebt), prüft `__runtime_ftoi_q88` mit `BN` :

- **Positiv (`Bit 15 == 0`):** 8× `SHR` → `RET` .
- **Negativ (`Bit 15 == 1`):** `NOT` → `ADD 1` ($|x|$) → 8× `SHR` ($|x| \gg 8$) → `NOT` → `ADD 1` ($-(|x| \gg 8)$) → `RET` .

---

9. Vollständiges Praxis-Beispiel: Heron-Wurzel $\sqrt{2}$ in `CLACK-F16` Assembler

Als Referenz für KIs zeigt dieses Beispiel, wie ein komplettes `CLACK-F16` -Programm (`X# = 0.5 * (X# + S# / X#)` für $S = 2.0$, Startwert $X_0 = 1.5$) in den Relais-Maschinencode übersetzt wird:

```
; ================================================================================
; HERON-ITERATION FUER SQRT(2.0) IN CLACK-F16 AUF DEM 16-BIT RELAISCOMPUTER
; Berechnet X_{k+1} = 0.5 * (X_k + 2.0 / X_k)
; Konstanten: 2.0 = 0x4000, 1.5 = 0x3F80, 0.5 = 0x3E00
; ================================================================================
ORG 0
    BA __main_start             ; [0..1] Boot-Sprung ueber I/O-Register 2 & 3

ORG 4
__main_start:
    LDA 0x4000                  ; S# = 2.0 (0x4000)
    STAM 920                    ; [920] = S#
    LDA 0x3F80                  ; X# = 1.5 (0x3F80)
    STAM 921                    ; [921] = X#
    LDA 0x3E00                  ; H# = 0.5 (0x3E00)
    STAM 922                    ; [922] = H#

    ; Schritt 1: Berechne T1# = S# / X# via __f16_div
    LDAM 920                    ; Lade S# (2.0)
    STAM __r0                   ; Operand A -> __r0
    LDAM 921                    ; Lade X# (1.5)
    STAM __r1                   ; Operand B -> __r1
    CALL __f16_div              ; Accu = 2.0 / 1.5 = 0x3F55 (~1.33203)

    ; Schritt 2: Berechne T2# = X# + (S# / X#) via __f16_add
    STAM __r1                   ; Rechts-Ergebnis (S#/X#) -> __r1
    LDAM 921                    ; Lade X# (1.5)
    STAM __r0                   ; Links-Operand X# -> __r0
    CALL __f16_add              ; Accu = 1.5 + 1.33203 = 0x4035 (~2.8359)

    ; Schritt 3: Berechne X_neu# = 0.5 * (X# + S# / X#) via __f16_mul
    STAM __r1                   ; Summe -> __r1
    LDAM 922                    ; Lade H# (0.5 = 0x3E00)
    STAM __r0                   ; 0.5 -> __r0
    CALL __f16_mul              ; Accu = 0.5 * 2.8359 = 0x3F6A (1.4140625 = sqrt(2)!)
    STAM 921                    ; Speichere neues X# = 0x3F6A

    ; Gib untere 8 Mantissen-Bits (0x6A = 106) auf Hardware Output Port [2] aus:
    AND 0x00FF
    STAM 2                      ; POKE 2, M

__halt:
    BA __halt                   ; Endlosschleife (HALT)
```

---

10. Checkliste für KI-Codegeneratoren (Vermeidung typischer Fehler)

1. **Keine verschachtelten `CALL` s in FPU-Routinen:** Das Hardware-Return-Register `SR` hat nur 1 Ebene. Nutze für Verkettungen wie `__f16_sub` → `__f16_add` immer `BA` .
2. **Kein Überschreiben von `__r0` bei geschachtelten Ausdrücken:** Wenn der rechte Teilausdruck selbst eine FPU-Routine aufruft, muss der bereits berechnete linke Teilausdruck in einer Temp-Zelle

(`952..959`) gepuffert werden, bis der rechte Teilausdruck fertig im `Accu` liegt!

3. **Patch-Offset exakt beachten:** Bei `STAM patch_addr` gefolgt von `LDAM varA` und `ADD 0` liegt das zu patchende Operandenwort des `ADD`-Befehls genau bei **`PC_von_STAM + 5`** (`STAM` belegt 2 Worte, `LDAM` belegt 2 Worte, der `ADD`-Opcode belegt 1 Wort → Wort `+5` ist der `ADD`-Operand!).
4. **`RET` ist 1 Wort, `NOT` / `SHR` / `SHL` sind 2 Worte:** Beim Berechnen von Wort-Adressen im RAM belegen `NOT` (`0x3540, 0x0000`), `SHR` (`0x3580, 0x0000`) und `SHL` (`0x35C0, 0x0000`) jeweils **2 Speicherworte**, während `RET` (`0x6000`) nur **1 Speicherwort** belegt.
5. **Kanonische Null (`0x0000`):** Prüfe vor dem Entpacken immer mit `AND 0x7FFF` und `BZ` auf `0.0`, da die implizite Eins (`OR 0x0100`) sonst aus `0.0` fälschlicherweise $1.0 \cdot 2^{-63}$ machen würde!